Rung Simulation Guide

Showing the Improvement Ladder
in the Vivado Simulator

TeraNova – FPGA Acceleration

Author	Kai Janssen (4994817)
Version	5 November 2026
Tools	Vivado 2025.1.1 (Vivado simulator, XSim), no licence needed
Files	`C:\PLD\sim\vivado`
Companion	*Vivado Simulation Guide* (how XSim works)

All cycle counts in this guide were checked by running the same VHDL with GHDL before the guide was written; they are identical to `sim\ladder.md`.

Contents

1 What you will show **2**

2 How the simulation is built **2**

2.1 What is the real design and what is testbench 3

3 Tonight: dry run **3**

4 Step by step **3**

4.1 Create the project 3

4.2 Run 4

4.3 The wave view 5

5 The demo, step by step **5**

5.1 The overview: five rungs, one start 5

5.2 Rung 0: why the baseline is slow 5

5.3 Rung 1: pipelined fetch 6

5.4 Rung 2: the writer fix 6

5.5 Rung 3: copy once, reuse ten times 6

5.6 Rung 4: the burst data path 6

5.7 Correctness 7

6 Running with the stress latency (L = 100) **7**

7 Backup: saved run and batch script **7**

7.1 Keep the finished run 7

7.2 The batch script 7

8 Questions your tutor may ask **8**

9 Troubleshooting this setup **9**

10 Files **10**

1 What you will show

One Vivado simulation that runs **all five rungs side by side**: five copies of the PL data path, each with the RTL of one rung, on the same 100 MHz clock, started by the same start pulse at the same moment. Every rung multiplies the same two 50×50 matrices and checks its own result. In the wave window you then see directly:

- how much sooner each rung finishes (one `busy` bar per rung, markers at every `done`);
- *why* it is faster, by zooming into the blocks: the partitioner state machine, the BRAM addresses, the handshakes between the blocks and the AXI write and read channels;
- that every rung still computes the right answer (the summary table in the Tcl Console).

Rung	Change	L = 20	vs prev.	L = 100	What you see in the waves
0	baseline (restored design)	101 186	–	101 586	4 cycles per element, 101 cycles per block pair
1	partitioner v5b	27 186	3.72×	58 576	a new BRAM address every cycle, 27 cycles per pair
2	writer fix	27 071	1.00×	27 151	writes no longer wait for each response (gain shows at L = 100: 2.15×)
3	partitioner v6	10 052	2.69×	18 052	one load phase, then a block pair every 6 cycles
4	burst data path	10 073	1.00×	18 153	PL reads A and B itself in 4 KB AXI bursts

Table 1: Expected PL run time (clock cycles from start to `done`, 10 ns each) for memory latency $L = 20$ (nominal) and $L = 100$ (stress). Rung 0 to rung 3 at $L = 20$: 10.06× faster.

Important. Rung 4 is *not* faster in this simulation, and that is correct: its gain is on the **CPU** side (no more 10 000 single CPU writes into the BRAMs and 2 500 uncached reads, estimated at 2.25 ms of the 2.35 ms end-to-end). A PL simulation cannot show CPU time. Say this before your tutor asks; it shows you understand where the time goes.

2 How the simulation is built

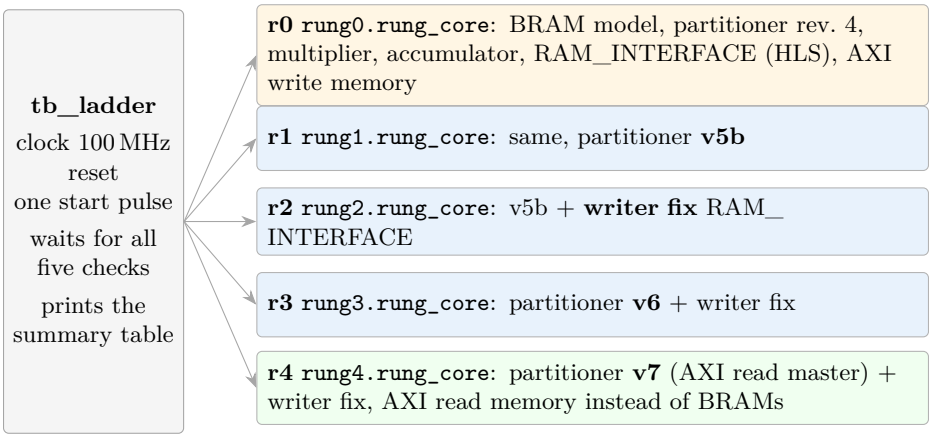


Figure 1: Structure of the simulation. Each rung is compiled into its own VHDL library, so the five partitioners and the two `RAM_INTERFACE` versions can keep the same entity names.

2.1 What is the real design and what is testbench

- **Design RTL, unchanged**: the multiplier (`MatMul5x5`, `MatMul5x5_IF_IN`), the accumulator, `matrix_to_ports`, the partitioners (`matrix_partitioner_B.vhd` and `variants\*_v5b/v6/v7.vhd`) and the **real Vitis-HLS RTL** of `RAM_INTERFACE` (`hls_rtl` and `steps\2_writer_fix\hls_rtl`). The scripts take them straight from `C:\PLD\sim`, the same files that `ladder.json` uses.
- **Testbench**: `rung_core` is a copy of `tb_chain_rtl.vhd` (rung 4: `tb_chain_axi.vhd`) with three changes: clock/reset/start come in through ports, the matrices come from a VHDL package instead of `a_b.hex`, and the result is read back from the memory model and compared with the exact product inside VHDL (instead of `c_out.hex` + Python). The memory models are the ones of `C:\PLD\sim` with a read-back port added.
- **Data**: the "hello" data set of `helloworld.c`: $A_{ij} = 50i + j + 1$, $B_{ij} = 50i + j + 2$.
- **Checks per rung**: all 2 500 words of $C = A \times B$ correct, exactly 100 output blocks (no lost or duplicated blocks), no unexpected AXI reads.

Why. The cycle count is measured exactly as in `tb_chain_rtl`: from the clock edge where the partitioner sees `start` until `done` of `RAM_INTERFACE`. The new testbench was run with GHDL before it was handed over and gives the same numbers as the ladder runner, for both latencies (Table 1). A check with a deliberately wrong expected value was also run, to confirm that the checker really reports errors.

3 Tonight: dry run

Do the whole demo once at home, so tomorrow only things you have already seen can happen.

- ☐ Laptop with **Vivado 2025.1.1**. No licence and no board needed.
- ☐ The folder `C:\PLD\sim\vivado` exists with `create_ladder_project.tcl`, `ladder_waves.tcl`, `run_ladder_batch.bat` and `src\`. The design files of all rungs must be in `C:\PLD\sim`: keep the branch `burst-datapath` checked out (the last rung; it contains the files of every earlier rung). It is checked out now.
- ☐ Run Section 4 once completely and write down how long compile and simulation take on your laptop.
- ☐ Walk through the demo of Section 5 once; place the zoom ranges yourself.
- ☐ Save a backup of the finished run (Section 7).
- ☐ Optional: run `run_ladder_batch.bat` once as a second, independent check.

Tip. The new files in `C:\PLD\sim\vivado` are not in git yet. If you want them on GitHub (and on Badr's laptop): `git add sim/vivado .gitignore` and commit them on `burst-datapath` (the branch you are on). The generated folders (`project`, `lib`, `batch`) are already in `.gitignore`.

4 Step by step

4.1 Create the project

1. Start Vivado 2025.1.1. Do not open a project.

2. In the **Tcl Console** at the bottom of the start page type:

```
source C:/PLD/sim/vivado/create_ladder_project.tcl
```

3. The console prints one line per rung and ends with `Project created: ...\project\teranova_ladder.xpr`. The project is open.

Check in the **Sources** window, Simulation Sources > sim_1:

- `tb_ladder` is in **bold** (the top) and has the instances `r0` to `r4` below it;
- in the Libraries tab there are `tn_common`, `rung0` ... `rung4` and `xil_defaultlib`;
- Design Sources is empty: this project only simulates. Vivado may show warnings about the empty design or about entities with the same name in different libraries; they do not matter.

The script made these settings for you (you can see them in Simulation Settings):

Top	`tb_ladder`
Generic	`LAT=20` (memory latency in cycles)
File type	all files `VHDL 2008`, each rung in its own library
Compile order	manual, in the order the files were added (Automatic Update, Manual Compile Order)
`xsim.simulate.runtime`	`-all`: run until the testbench ends (about 1.09 ms simulated)
`xsim.simulate.log_all_signals`	`true`: every signal is recorded from time 0
`xsim.simulate.tcl.post`	`ladder_waves.tcl`: builds the wave view after the run

4.2 Run

1. Flow Navigator SIMULATION > Run Simulation > Run Behavioral Simulation.
2. Vivado compiles about 75 files (five rungs), elaborates and runs to the end. This takes a few minutes; the status bar at the top right shows the step.
3. When it is done, the **Tcl Console** shows the summary:

```
=================== TeraNova improvement ladder (Vivado XSim) ====================
AXI memory latency L = 20 cycles, clock 100 MHz (10 ns per cycle)
rung                              PL cycles   PL time (us)   vs previous   vs baseline   multiplier busy   result
rung 0  baseline                     101186        1011.86         1.00x         1.00x            4.94 %   PASS (2500/2500)
rung 1  partitioner v5b               27186         271.86         3.72x         3.72x           18.39 %   PASS (2500/2500)
rung 2  writer fix                    27071         270.71         1.00x         3.73x           18.46 %   PASS (2500/2500)
rung 3  partitioner v6 (reuse)        10052         100.52         2.69x        10.06x           64.28 %   PASS (2500/2500)
rung 4  burst data path               10073         100.73         0.99x        10.04x           64.15 %   PASS (2500/2500)
ALL RUNGS PASS: every rung computed all 2500 result words of C = A x B correctly
==================================================================================
```

4. A wave window tab **TeraNova ladder** opens with the groups of Section 4.3 and five markers `rung 0 done` ... `rung 4 done`. Press Zoom Fit.

Important. If the wave view did not appear (the console says `ladder_waves: ...`), type `source C:/PLD/sim/vivado/ladder_waves.tcl` in the Tcl Console. Because all signals were logged, the waves still show the whole run. If the simulation stopped at 1000 ns, the run-time setting was not applied: press Run All.

4.3 The wave view

Group	Contents
Overview	`rst`, `start`; `busy rung 0..4` (high from start until `done`: *this bar is the PL run time*); `cycles rung 0..4` (live cycle counters); `all results checked`
Rung n > Partitioner	`state` (shown by name), BRAM address A (rungs 0–3) or the AXI read channel (rung 4), the block counters `i_blk`, `j_blk`, `p_blk`
Rung n > Handshakes	pair valid/ready (partitioner → multiplier), product valid (multiplier → accumulator), block valid/read (accumulator → writer), output blocks written
Rung n > Writer	the AXI write channel of `RAM_INTERFACE`: AW, W, B handshakes, `awaddr`, `awlen`, and `done`

Collapse the five rung groups at first (triangle in front of the name), so only the overview is visible. Open one rung group at a time during the demo.

5 The demo, step by step

Times below are simulation times. To zoom to a range, drag in the time ruler above the waves, or zoom in with Ctrl + mouse wheel around the point. Start, reset release and the start pulse are at the very beginning: the start pulse is at about 0.17 µs.

5.1 The overview: five rungs, one start

Zoom Fit. The five `busy` bars start together and end at the markers:

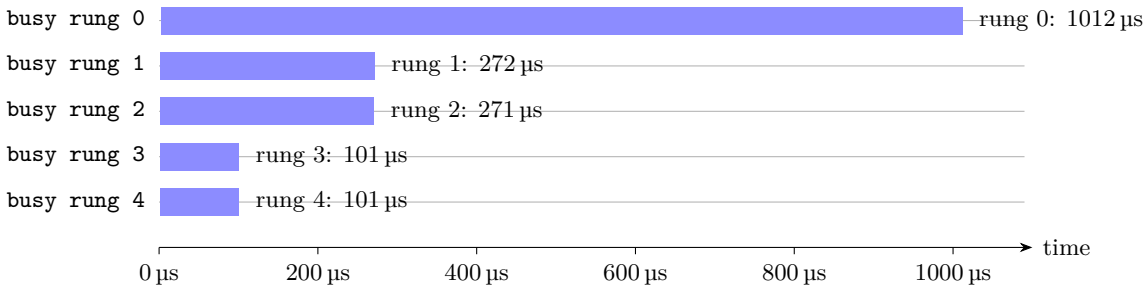


Figure 2: Sketch of the overview (to scale). The real view is the Vivado wave window.

- Click on a marker label, or on the falling edge of a `busy` bar: the `cycles rung n` value in the Value column is the PL run time in clock cycles (101 186, 27 186, . . .).
- To measure it in time: put a marker on the rising edge of `start`, then click on the end of a `busy` bar. The distance between cursor and marker is the run time.

What to say. "These are five versions of our PL data path running in parallel in one simulation, all with the same input and the same start. Each bar is how long that version needs for one 50 by 50 matrix multiplication. The baseline needs 1.01 milliseconds, the last versions 0.10 milliseconds: ten times faster in the PL, and all five produce exactly the right result."

5.2 Rung 0: why the baseline is slow

Open **Rung 0** > **Partitioner** and **Handshakes**. Zoom to about 1.0–2.3 µs.

- `state` goes `ISSUE_ADDR` → `WAIT1` → `WAIT2` → `CAPTURE` for every element: the **BRAM address changes only every 40 ns = 4 cycles**.
- A block pair needs 25 elements: `pair valid` pulses every **1.01 µs = 101 cycles**.
- `pair ready` is almost always high: the multiplier, which needs 5 cycles per pair, waits about 95 % of the time ("multiplier busy 4.94 %" in the table).

What to say. "The BRAMs have a read latency of one cycle, but the partitioner spends four cycles per element. The two wait states do nothing. So the expensive single-cycle multiplier is idle 95 percent of the time."

5.3 Rung 1: pipelined fetch

Open **Rung 1 > Partitioner** and **Handshakes**, same zoom range.

- `state` is `FETCH` while the 25 elements are read, then briefly `OUTPUT`: a new BRAM address **every cycle (10 ns)**; each element is captured two edges after its address.
- `pair valid` every **270 ns = 27 cycles** instead of 101. Result: 27 186 cycles, 3.72×.

Tip: put rung 0's and rung 1's `BRAM address A` next to each other (drag them into one group) and zoom to about 50 ns per division: the staircase of rung 0 has steps 4 times as long.

5.4 Rung 2: the writer fix

At $L = 20$ rung 2 is only 115 cycles faster, so first show *what* changed. Open **Writer** of rung 1 and of rung 2 and zoom to about 3.0–3.6 µs.

- Every output block (5×5 values) is written as 5 bursts of 5 beats (`awlen` = 4), one per row of the block.
- **Rung 1**: the next `awvalid` comes only after the `bvalid` of the previous burst: one burst every 330 ns. The writer waits for every write response.
- **Rung 2**: the 5 `awvalid`s follow each other every 60 ns, while the `bvalid`s arrive later: the responses overlap with new writes.

Then show where it matters, with a higher memory latency (Section 6): at $L = 100$ rung 1 needs 58 576 cycles because the writer becomes the bottleneck, rung 2 still 27 151 (2.15×).

What to say. "With a realistic DDR latency the old writer would have eaten most of the gain of rung 1. The fix makes the next rungs independent of a latency we have not measured on the board yet."

5.5 Rung 3: copy once, reuse ten times

Open **Rung 3 > Partitioner** and **Handshakes**; zoom to 0–30 µs, then into 25–27 µs.

- From the start until about 25.2 µs `state` is `LOAD`: A and B are copied *once* from the BRAMs into 25 on-chip banks per matrix, one element of each per cycle (about 2 500 cycles).
- After that `state` moves through `RD`, `CAP` and `OUTPUT` for every block pair, and `pair valid` comes every **6 cycles**: a whole 5×5 block is read from the 25 banks in one cycle.
- The multiplier is now busy 64 % of the time. Total 10 052 cycles: 2.69× faster than rung 2, 10.06× faster than the baseline.

5.6 Rung 4: the burst data path

Open **Rung 4 > Partitioner**; zoom to 0–26 µs.

- There are no BRAMs any more: the partitioner itself reads A and B through an **AXI4 read master**. `AXI read ARLEN` = 255 means bursts of 256 beats of 128 bits = 4 KB, the

maximum before a 4 KB boundary; the last burst of each matrix is 226 beats. `AXI read RVALID` is high 99 % of the load phase: the read channel is used at full speed.
- The load phase ends at about 25.4 µs; from there it is the same compute phase as rung 3. Total 10 073 cycles: equal to rung 3.

What to say. "In the PL, rung 4 costs the same as rung 3. What it removes is CPU work that this simulation does not contain: today the processor writes 10 000 words into the BRAMs one by one and reads 2 500 results back uncached. That is estimated at over 2 of the 2.35 milliseconds end-to-end. With the burst data path the CPU only flushes and invalidates its cache."

5.7 Correctness

Scroll the Tcl Console back to the summary: every rung says `PASS (2500/2500)`. Each rung compared all 2 500 words of its result with the exact product, and counted exactly 100 output blocks.

6 Running with the stress latency (L = 100)

The memory latency is the generic `LAT` of `tb_ladder`. In the Tcl Console:

```
close_sim
set_property generic {LAT=100} [get_filesets sim_1]
launch_simulation
```

(GUI: Simulation Settings > Compilation > Generics/Parameters, then Run Simulation.) Expected summary: 101 586 / 58 576 / 27 151 / 18 052 / 18 153 cycles, all PASS. Set it back with `{LAT=20}`.

Tip. Running both latencies takes two full simulations. If time in the meeting is short, run $L = 20$ live and have the $L = 100$ result ready from the batch script (Section 7.2).

7 Backup: saved run and batch script

7.1 Keep the finished run

After the dry run, copy these two files to a safe folder (for example `C:\PLD_demo`):

- `C:\PLD\sim\vivado\project\teranova_ladder.sim\sim_1\behav\xsim\tb_ladder_behav.wdb` (the recorded waves);
- `C:\PLD\sim\vivado\ladder.wcfg` (the wave layout, saved by `ladder_waves.tcl`).

In the meeting you can then open the result without simulating: in Vivado Flow > Open Static Simulation and choose the `.wdb`, then File > Simulation Waveform > Open Configuration and choose `ladder.wcfg`. (Static means: you can look and measure, but not run further.)

7.2 The batch script

`run_ladder_batch.bat` runs the same simulation without a project, with the three simulator steps directly (`xvhdl`, `xelab`, `xsim`). Double-click it, or from a command prompt:

```
cd /d C:\PLD\sim\vivado
run_ladder_batch.bat            :: L = 20, summary in the window
run_ladder_batch.bat 100        :: L = 100
run_ladder_batch.bat 20 gui     :: and open the XSim GUI with the ladder wave view
```

It looks for Vivado in `C:\Xilinx\2025.1.1\Vivado`; if Vivado is somewhere else, start it from the Vivado 2025.1.1 Tcl Shell or change the `SETTINGS` line at the top. Logs go to `C:\PLD\sim\vivado\batch`.

8 Questions your tutor may ask

Question	Answer
Is this the real RTL?	Yes: the partitioners, multiplier and accumulator VHDL of the design, and the RTL that Vitis HLS generated for `RAM_INTERFACE`. Only the surroundings are models: a BRAM model (read latency 1, like the BRAMs in the block design without output register), an AXI4 write slave and, for rung 4, an AXI4 read slave.
Behavioural or timing simulation?	Behavioural (RTL). It shows cycle counts at 100 MHz, not whether the design meets timing. Timing closure is checked with the implementation on the licensed laptop (WNS ≥ 0 per rung, see the board guide).
What is L?	The latency of the AXI memory: cycles from the last write beat to the write response, and from a read address to the first read data. Real PS/PL DDR latency is not measured yet, so two values are simulated: 20 (nominal) and 100 (stress).
Why only one data set?	The live demo uses the hello data of `helloworld.c` because the expected result can be computed exactly in VHDL. The full GHDL regression in `C:\PLD\sim` also runs random full-range 64-bit data, a slow writer with back-pressure and a restart without reset; all pass, with the same cycle counts.
Does the cycle count depend on the data?	No. The data path is fixed: the counts are identical for the hello and the random data set (checked in the GHDL runs).
Why is rung 4 not faster?	In the PL it is the same compute phase as rung 3; its gain is the CPU load and read-back that disappear (Section 5, "Rung 4"). Measured on the board with the rung-4 application.
Why is the multiplier still idle 36 %?	Rung 3/4: the writer needs about 75 cycles per output block while the multiplier could do one in 60. Next candidates are longer write bursts and a second read port (*simulation report*, "next rungs").
Why not pipeline the multiplier?	It is busy only 5 of every 6 cycles in rung 3 and already does a whole 5×5 block product in one cycle. Pipelining it pays off only after the writer and the loading are faster; but it would also allow a higher clock, which matters for timing.
How do you know the simulation is right?	Every rung checks all 2 500 result words and the number of output blocks; the same RTL was simulated independently with GHDL with identical cycle counts; and the board measurements (board guide) will compare the PL time with these numbers.

9 Troubleshooting this setup

Problem	What to do
`source` says "Missing source files"	The script lists which files. Usually an earlier branch is checked out, which does not have the partitioners of the later rungs yet. `git checkout burst-datapath` (the files in `sim\vivado` are untracked and stay where they are).
Compile step fails	Read the first error in the Tcl Console or in `project\teranova_ladder.sim\sim_1\behav\xsim\compile.log`. If it is in `RAM_INTERFACE_gmem_m_axi.vhd` and mentions a conditional expression (`when ... else` in a constant), a non-patched HLS file was copied into `sim\hls_rtl`: use the patched one (`SIM PATCH` comment), or set that file's type to `VHDL 2019`.
Elaborate step fails: `rung_core` not found	The compile order is wrong. In Sources > Compile Order `tn_pkg.vhd` must be first and `tb_ladder.vhd` last. Simplest: `close_project`, then `source` the create script again.
Simulation stops at 1000 ns	Press Run All.
Wave view empty or missing	`source C:/PLD/sim/vivado/ladder_waves.tcl`. Signals with no waveform: they were not logged; Restart, `log_wave -r /`, Run All.
Summary shows FAIL	Do not show it. Something in `C:\PLD\sim` changed since the GHDL check; compare with `git diff`, and re-run `python run_ladder.py` in `C:\PLD\sim`.
Vivado very slow / out of memory	Close other programs. In the settings set `xsim.simulate.log_all_signals` to `false`; then add the overview signals before Run All (or run `ladder_waves.tcl` right after launch and Restart, Run All).
Files locked when relaunching	Close Simulation first; if needed end `xsimk.exe` in the Task Manager.

10 Files

File (in C:\PLD\sim\vivado)	What it is
create_ladder_project.tcl	creates project\teranova_ladder.xpr: copies each rung's RTL into lib\rungN, one library per rung, settings
ladder_waves.tcl	builds the wave view, places the markers, saves ladder.wcfg
run_ladder_batch.bat, batch_gui.tcl	the same simulation without a project
src\tb_ladder.vhd	top: five rungs side by side, summary table
src\rung_core_bram.vhd	one rung (0–3): copy of tb_chain_rtl.vhd with ports and a built-in check
src\rung_core_axi.vhd	rung 4: copy of tb_chain_axi.vhd, same changes
src\axi_mem_model_v.vhd	AXI write memory of C:\PLD\sim with a read-back port
src\axi_rd_mem_model_v.vhd	AXI read memory of C:\PLD\sim with built-in data
src\tn_pkg.vhd	hello data and the exact expected result
practice\	the counter exercise of the Vivado Simulation Guide
project\, lib\, batch\	generated, can be deleted, in .gitignore